UNIVERSIDAD TECNOLÓGICA ECOTEC

FACULTAD DE INGENIERÍAS, ARQUITECTURA Y CIENCIAS DE LA NATURALEZA

INGENIERÍA EN SISTEMAS INTELIGENTES

PROGRAMACIÓN DE SISTEMAS WEB

PROFESOR

GREGORIO CRUZ G

AUTOR

TNLGO. GEORGE ELIEZER LUCAS CANTOS

TEMA: ACTIVIDAD INTEGRADORA 3. MESA DE AYUDA: SISTEMA WEB DINÁMICO EN PHP CON MYSQL, SENTENCIAS PREPARADAS Y CONTROL DE VERSIONES

REPOSITORIO PÚBLICO EN GITHUB:

https://github.com/glucas/AI3_DisenoWeb_GeorgeLucas

GUAYAQUIL – ECUADOR

8 DE NOVIEMBRE DE 2026

DESARROLLO DE UNA MESA DE AYUDA CON PHP Y MYSQL

Para esta actividad desarrollé una **mesa de ayuda**: un módulo web donde el personal de una institución reporta incidencias de soporte técnico (equipos que no encienden, impresoras, red, accesos) y el área de sistemas las consulta en una tabla. Elegí este caso porque es el tipo de solicitud que atiendo en mi trabajo de soporte técnico. El sistema cubre el registro (Create) y la consulta (Read) sobre la base **db_sistema_web**, y está organizado con el patrón Modelo-Vista-Controlador.

1. JUSTIFICACIÓN TÉCNICA

1.1. MySQLi y PDO: diferencias

PHP tiene dos extensiones para conectarse a MySQL. Investigué sus diferencias en tres aspectos: arquitectura, rendimiento y seguridad.

Aspecto	MySQLi	PDO (PHP Data Objects)
Arquitectura	Controlador hecho solo para MySQL y MariaDB. Se puede usar con funciones (mysqli_query) o con objetos.	Interfaz única orientada a objetos. Debajo usa un controlador por motor (pdo_mysql, pdo_pgsql, pdo_sqlite, pdo_sqlsrv, etc.).
Portabilidad	Ninguna: cambiar de motor obliga a reescribir el acceso a datos.	Alta: para cambiar de motor normalmente basta con modificar la cadena DSN.
Sentencias preparadas	Marcadores posicionales (?) y bind_param() con una cadena de tipos («ssis»).	Marcadores posicionales o con nombre (:correo); los valores pueden pasarse en un arreglo a execute().
Errores	Lanza mysqli_sql_exception por defecto desde PHP 8.1.	Lanza PDOException por defecto desde PHP 8.0 (ERRMODE_EXCEPTION).
Rendimiento	Acceso directo a la API de MySQL; en teoría, algo más liviano.	La capa de abstracción añade un costo mínimo, despreciable frente al tiempo de red y de la consulta.
Seguridad	Equivalente si se usan sentencias preparadas.	Equivalente si se usan sentencias preparadas. En MySQL, PDO emula la preparación salvo que se desactive ATTR_EMULATE_PREPARES.

Tabla 1. Comparación entre MySQLi y PDO (elaboración propia con base en el manual de PHP y PHP.Watch).

1.2. Selección: PDO

Elegí PDO por tres razones técnicas:

- **Portabilidad.** Las instituciones públicas a menudo combinan motores de bases de datos. Con PDO, el modelo puede trabajar con PostgreSQL o SQL Server cambiando solo la cadena de conexión de `config/conexion.php`.
- **Abstracción.** Los parámetros con nombre (`:solicitante`, `:correo`...) se asocian por nombre y no por posición, lo que evita errores al agregar columnas. Además, la interfaz de objetos encaja con el modelo MVC: la clase `IncidenciaModelo` recibe un objeto PDO y no depende de funciones globales.

- **Manejo de excepciones.** Con `PDO::ERRMODE_EXCEPTION`, cualquier falla de conexión o de SQL interrumpe la ejecución con una `PDOException`, que capturo con try-catch. Así se registra el detalle en el log y el usuario recibe un mensaje general, sin que se muestren credenciales ni nombres de tablas.

1.3. Medidas de seguridad implementadas

Sentencias preparadas. El modelo nunca concatena datos en el SQL. El INSERT usa marcadores con nombre y los valores viajan en el arreglo de `execute()`. Además desactivé la emulación (`ATTR_EMULATE_PREPARES => false`), de modo que MySQL recibe primero la instrucción con «?» y después los datos por separado. El registro de consultas del servidor durante las pruebas lo muestra:

```
Prepare  INSERT INTO incidencias (solicitante, correo, id_area, tipo, prioridad, descripcion)
         VALUES (?, ?, ?, ?, ?, ?)
Execute  INSERT INTO incidencias (...) VALUES ('Jorge Bajaña', 'jbajana@institucion.gob.ec', '4',
         'Software', 'Alta', 'Al abrir el sistema de asistencia aparece el error ...')
```

Registro general (general log) de MariaDB durante las pruebas.

Escape de salidas (XSS). La función `h()` de `app/ayudantes.php` aplica `htmlspecialchars($texto, ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML5, 'UTF-8')`. Todas las vistas la usan para cada dato de la base o del usuario, incluidos los valores que se devuelven al formulario cuando hay errores. En la descripción, primero se escapa y después `nl2br()` agrega los saltos de línea, en ese orden, para no reabrir la puerta al HTML. Como segunda capa, la cabecera Content-Security-Policy bloquea scripts que no provengan del propio sitio.

Conexión sin exponer credenciales. `config/conexion.php` lee los datos de acceso de `credenciales.php`, archivo excluido en `.gitignore`; en GitHub solo está la plantilla `credenciales.ejemplo.php`. Si la conexión falla, el bloque catch envía el mensaje original a `error_log()` y el usuario ve «El servicio de base de datos no está disponible». Además, `display_errors` está desactivado.

```
try {
    $conexion = new PDO($dsn, $cfg['usuario'], $cfg['contrasena'], $opciones);
} catch (PDOException $error) {
    error_log('[mesa-ayuda] Fallo de conexión: ' . $error->getMessage());
    throw new RuntimeException('El servicio de base de datos no está disponible ...');
}
```

Validación en cliente y servidor. En el navegador, los atributos HTML5 (required, pattern, minlength, maxlength, type="email") y `formulario.js` avisan antes de enviar y muestran un contador de caracteres. En el servidor, la clase `Validador` revisa todo de nuevo: el nombre solo admite letras (3 a 80); el correo, un formato válido; y el área, el tipo y la prioridad se comparan con listas blancas, de modo que un valor manipulado como «Cafetera» o el área 99 se rechaza. La descripción debe tener de 10 a 500 caracteres.

Otras medidas: token CSRF verificado con `hash_equals()`; redirección 303 después de guardar, para que recargar la página no duplique el registro; un usuario de base de datos con solo SELECT e INSERT en las pruebas; y archivos `.htaccess` que impiden abrir `config/`, `app/`, `sql/` y `.git/` desde el navegador.

1.4. Pruebas realizadas

Automaticé 38 pruebas con un navegador real (Playwright) contra el sistema en ejecución; todas pasaron. Las de seguridad fueron:

Ataque o caso	Dato enviado	Resultado
Inyección SQL	'); DROP TABLE incidencias; --	Guardado como texto; la tabla sigue intacta.
Apóstrofo legítimo	O'Connor	Se guarda y se muestra correctamente.
XSS almacenado	<script>alert('XSS')</script> <img onerror=...>	Se muestra como texto; no se ejecuta nada.
XSS reflejado	<b>x en el nombre	Rechazado y devuelto escapado al formulario.
Valores fuera de lista	tipo «Cafetera», área 99, prioridad «Urgente»	HTTP 422 con un mensaje por campo.
Sin token CSRF	POST directo	HTTP 400 y nada insertado.
Base de datos caída	Contraseña del usuario cambiada	HTTP 503 con mensaje genérico, sin datos técnicos.
Acceso a carpetas internas	/config/credenciales.php, /.git/config	403 y 404.

Tabla 2. Pruebas de seguridad.

2. ARQUITECTURA DEL SISTEMA

Usé la arquitectura **Modelo-Vista-Controlador**, con un controlador frontal. Todas las peticiones entran por `index.php`, que lee el parámetro `accion` y llama al método correspondiente de `IncidenciaControlador`.

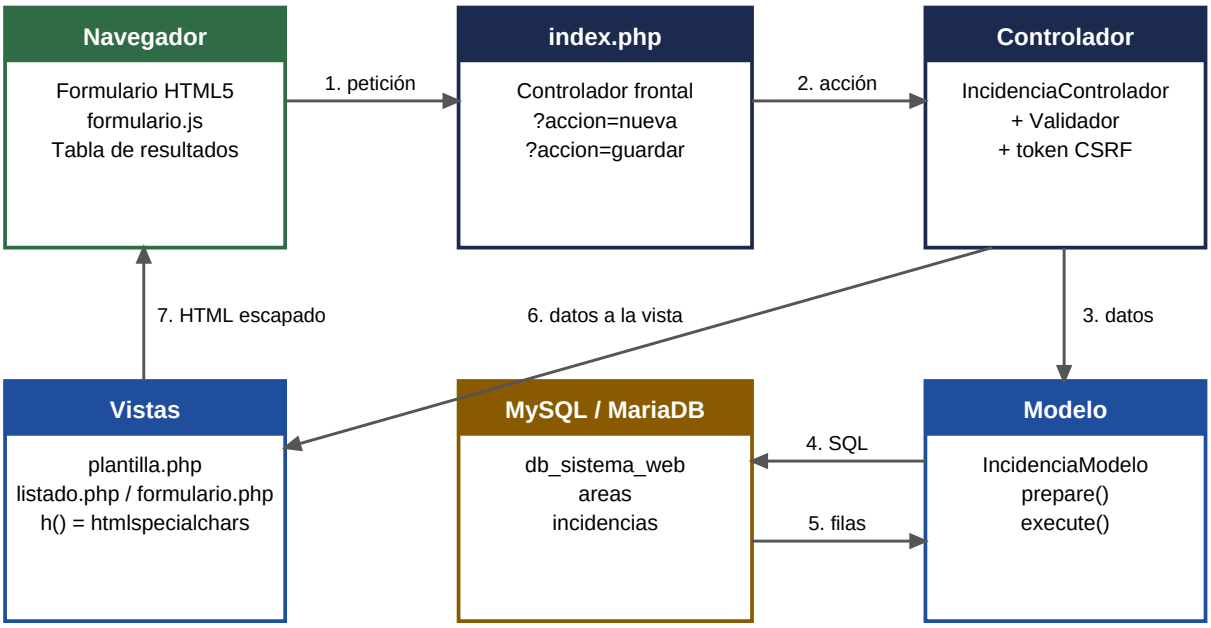


Figura 1. Flujo de datos entre el cliente, el código PHP y la base de datos (elaboración propia).

Registro de una incidencia:

- El usuario llena el formulario; `formulario.js` revisa los campos y, si están bien, el navegador envía un POST a `index.php?accion=guardar`.
- El controlador frontal llama a `guardar()`, que verifica el token CSRF y pasa los datos al `Validador`.
- Si hay errores, se vuelve a mostrar el formulario con los mensajes (HTTP 422). Si no, el controlador entrega los datos limpios al modelo.
- El modelo ejecuta `prepare()` y `execute()`; MySQL aplica además sus propias reglas (ENUM, llave foránea, NOT NULL) y asigna el id y la fecha.
- El controlador guarda el mensaje «Incidencia INC-0007 registrada» en la sesión y responde con una redirección 303 al listado.

Consulta: el modelo ejecuta un SELECT preparado con JOIN entre `incidencias` y `areas`; la vista `listado.php` recorre las filas y escapa cada dato con `h()` antes de construir la tabla.

Modelo de datos. La tabla principal **incidencias** tiene 10 campos y se relaciona con el catálogo **areas**:

Campo	Tipo	Detalle
id_incidencia	INT UNSIGNED	Clave primaria AUTO_INCREMENT. Se muestra como INC-0001.
solicitante	VARCHAR(80)	NOT NULL.
correo	VARCHAR(120)	NOT NULL.
id_area	SMALLINT UNSIGNED	Llave foránea a areas(id_area).
tipo	ENUM	Hardware, Software, Red, Impresora o Accesos.
prioridad	ENUM	Baja, Media, Alta o Crítica. DEFAULT 'Media'.
descripcion	VARCHAR(500)	NOT NULL.
estado	ENUM	Abierta, En proceso o Resuelta. DEFAULT 'Abierta'.
fecha_creacion	TIMESTAMP	Auditoría: DEFAULT CURRENT_TIMESTAMP.
fecha_actualizacion	TIMESTAMP	Auditoría: ON UPDATE CURRENT_TIMESTAMP.

Tabla 3. Estructura de la tabla incidencias (script en sql/db_sistema_web.sql).

3. EVIDENCIAS DE EJECUCIÓN

Las capturas son del sistema funcionando en un entorno local con Apache 2.4, PHP 8.3, MariaDB 10.11 y phpMyAdmin 5.2, el mismo conjunto de herramientas que incluye XAMPP.

3.1. Estructura de la tabla en phpMyAdmin

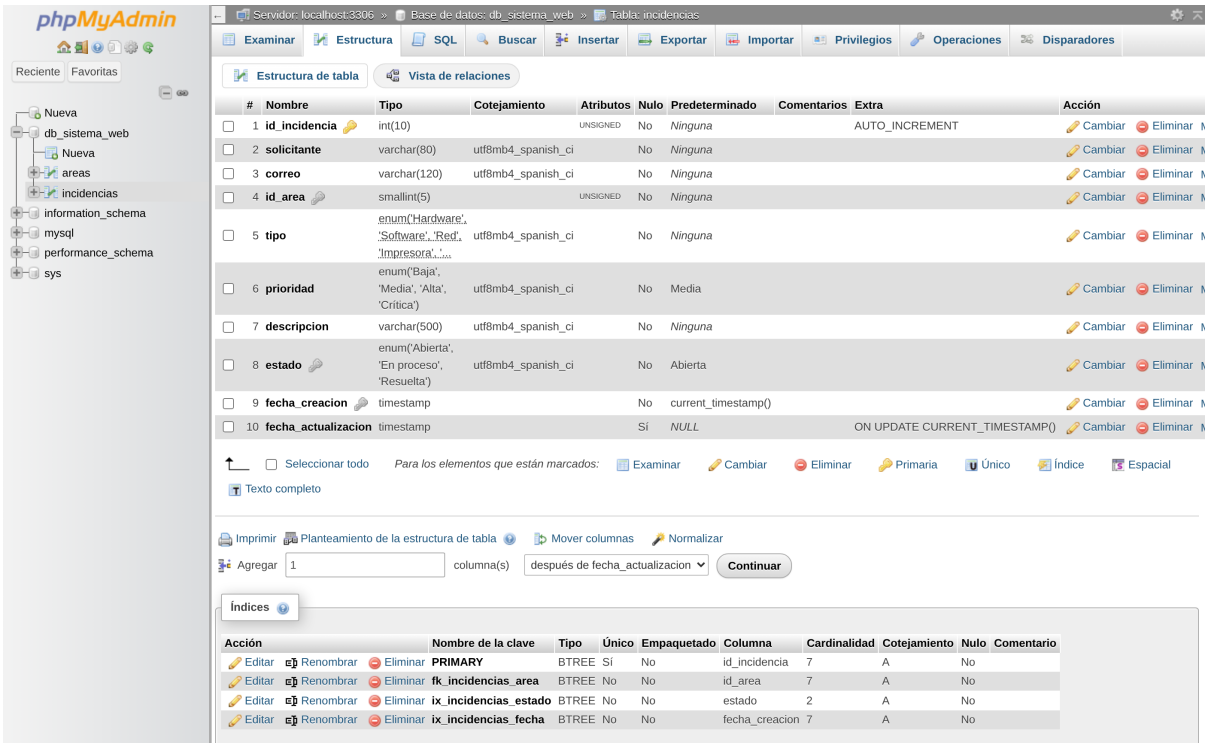


Figura 2. Estructura de la tabla incidencias en phpMyAdmin.

Se ven los 10 campos con sus tipos. id_incidencia tiene la llave dorada de clave primaria y AUTO_INCREMENT. Los ENUM limitan tipo, prioridad y estado a valores conocidos, y prioridad y estado tienen valores predeterminados (Media y Abierta). Los dos campos de auditoría usan TIMESTAMP: fecha_creacion se llena sola con current_timestamp() y fecha_actualizacion cambia sola con ON UPDATE. En «Índices» aparecen la clave primaria, el índice de la llave foránea hacia areas y dos índices para filtrar por estado y por fecha.

3.2. Código con sentencias preparadas

app/Modelos/IncidenciaModelo.php

```
35      /**
36       * Inserta una incidencia validada con una sentencia preparada.
37       * Los datos se envían aparte de la instrucción SQL, por eso no pueden alterarla.
38       */
39      public function registrar(array $incidencia): int
40      {
41          $sentencia = $this->bd->prepare(
42              'INSERT INTO incidencias (solicitante, correo, id_area, tipo, prioridad, descripcion)
43               VALUES (:solicitante, :correo, :id_area, :tipo, :prioridad, :descripcion)'
44          );
45          $sentencia->execute([
46              ':solicitante' => $incidencia['solicitante'],
47              ':correo'      => $incidencia['correo'],
48              ':id_area'     => $incidencia['id_area'],
49              ':tipo'        => $incidencia['tipo'],
50              ':prioridad'   => $incidencia['prioridad'],
51              ':descripcion' => $incidencia['descripcion'],
52          ]);
53          return (int) $this->bd->lastInsertId();
54      }
```

Figura 3. Método registrar() de app/Modelos/IncidenciaModelo.php.

La línea 41 llama a **prepare()** con un INSERT que solo tiene marcadores con nombre (líneas 42-43); ningún dato del usuario forma parte del texto SQL. Entre las líneas 45 y 52, **execute()** recibe un arreglo que asocia cada marcador con su valor ya validado. La línea 53 devuelve el id generado, que el controlador usa para mostrar «INC-0007».

3.3. Formulario en ejecución con datos de prueba

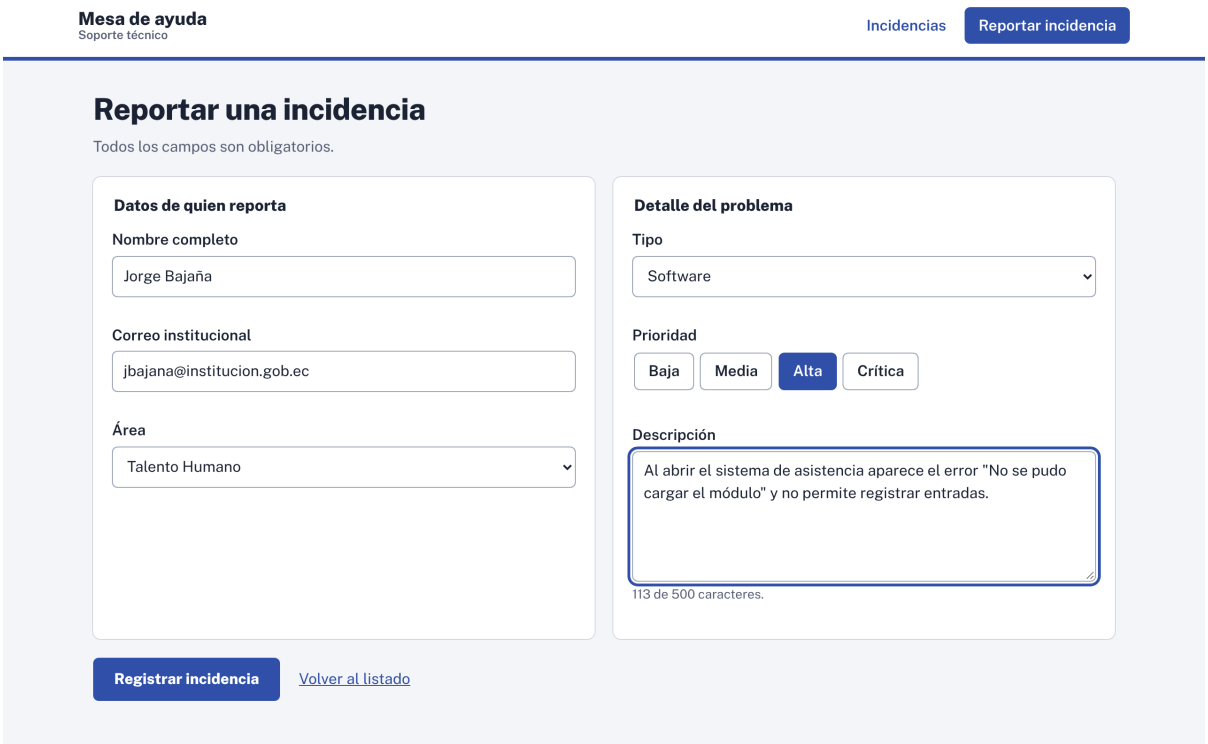


Figura 4. Formulario de registro con una incidencia de prueba.

El formulario agrupa los campos en dos bloques: quién reporta y detalle del problema. La prioridad se elige con botones de opción (aquí, «Alta») y la descripción muestra un contador

de caracteres (113 de 500). Área y tipo son listas desplegables, de modo que el usuario no puede escribir valores libres; aun así, el servidor vuelve a validarlos.

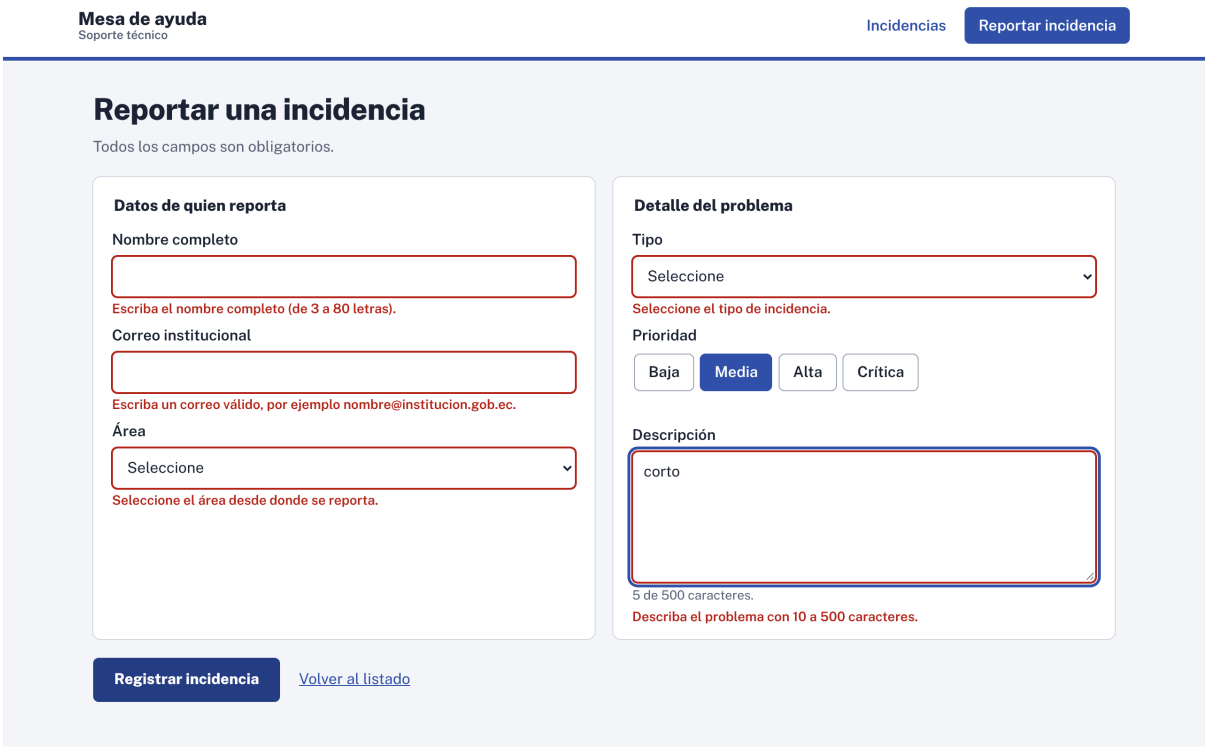


Figura 5. Validación en el navegador al intentar enviar el formulario incompleto.

Al enviar el formulario vacío, con una descripción de solo 5 caracteres, la validación del cliente bloquea el envío, marca en rojo cada campo y explica qué falta. La prioridad no aparece marcada porque viene seleccionada por defecto en «Media».

3.4. Tabla web con los registros insertados

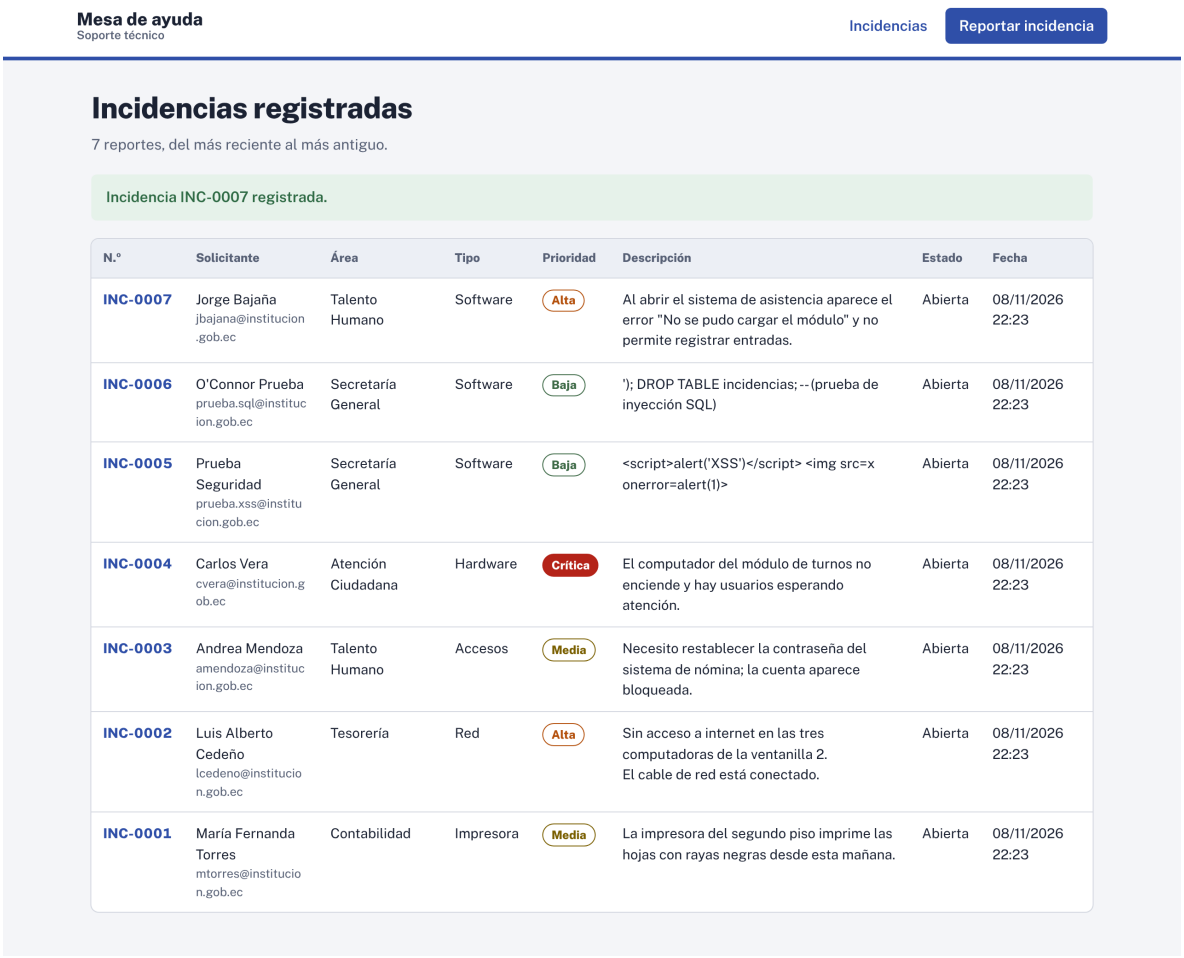


Figura 6. Listado de incidencias después de registrar INC-0007.

Después de guardar, la redirección lleva al listado y aparece el aviso verde «Incidencia INC-0007 registrada». La tabla muestra las 7 incidencias, de la más reciente a la más antigua, con el área obtenida por el JOIN y el estado «Abierta» asignado por la base. Las filas INC-0005 e INC-0006 son pruebas de ataque: el código <script> y la etiqueta <img onerror> aparecen como texto, porque h() los convirtió en entidades HTML; y el «DROP TABLE» se guardó como una descripción más, porque la sentencia preparada lo trató como dato. En INC-0002 se conserva el salto de línea de la descripción.

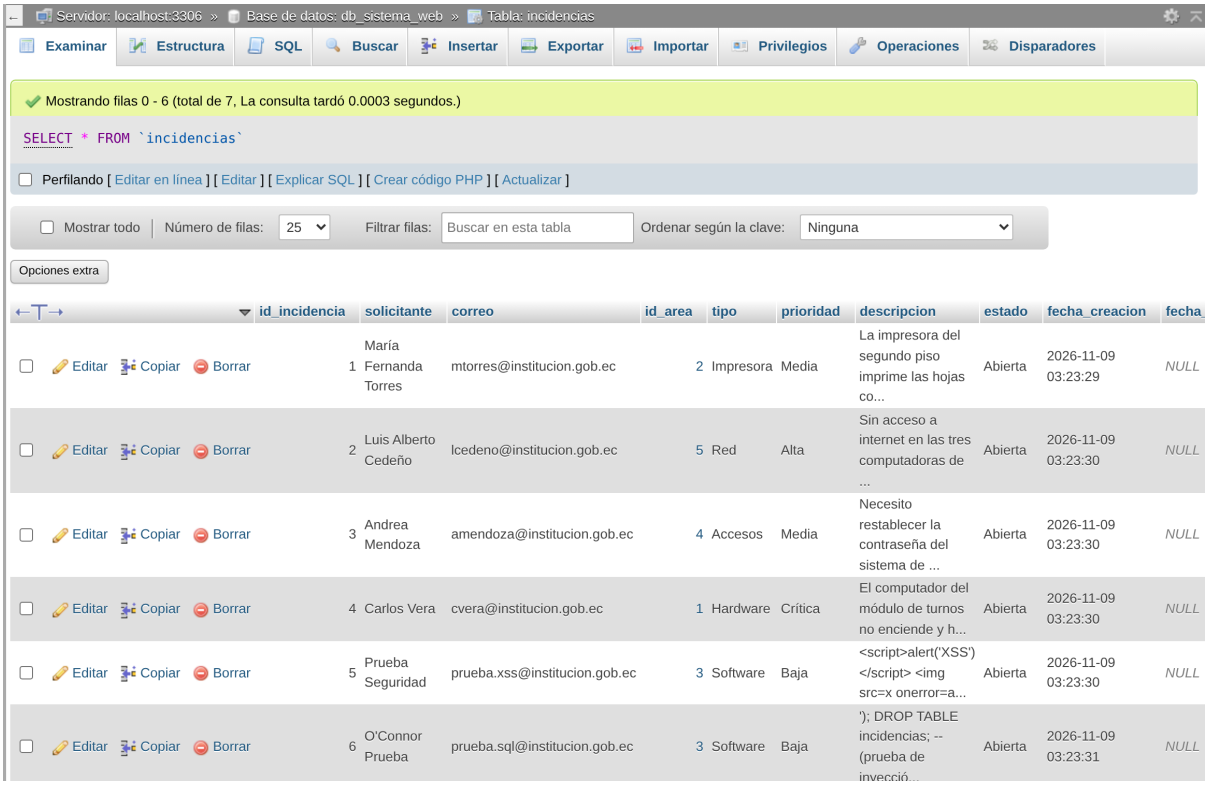


Figura 7. Los mismos registros en phpMyAdmin.

phpMyAdmin confirma que las 7 filas existen en la base, con su id autoincremental, el id_area numérico (la vista web muestra el nombre gracias al JOIN) y la fecha_creacion llenada automáticamente.

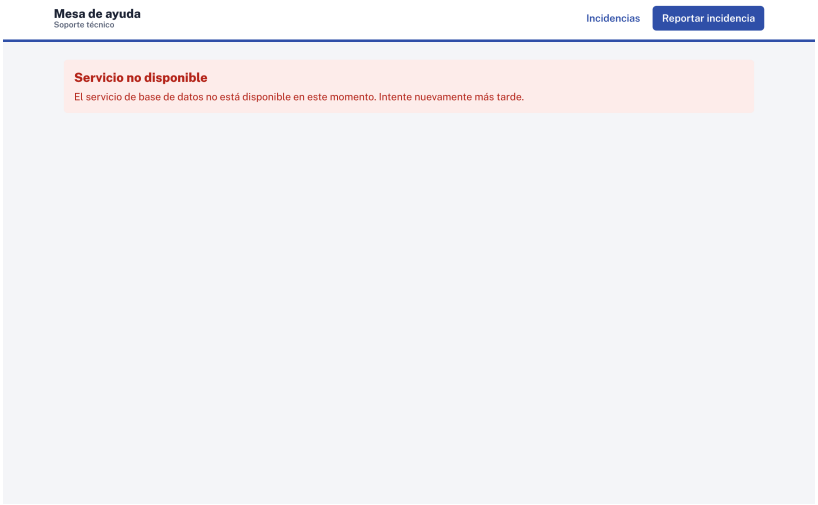


Figura 8. Respuesta del sistema cuando la base de datos no acepta la conexión.

Para comprobar el try-catch cambié la contraseña del usuario de la base. La aplicación respondió con HTTP 503 y un mensaje general; el texto real del error (Access denied for user...) solo quedó en el log de Apache.

3.5. Historial de commits

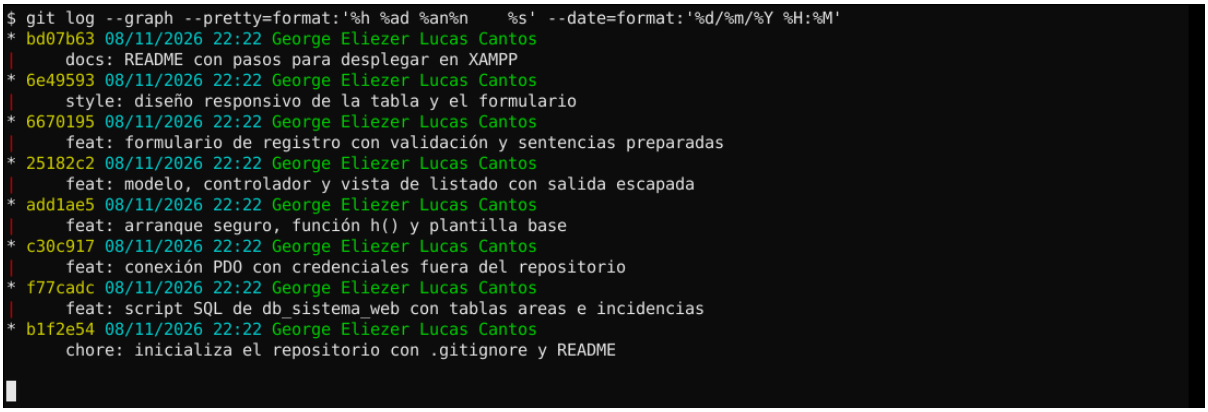


Figura 9. Historial del repositorio obtenido con git log --graph.

El trabajo quedó registrado en ocho commits atómicos con prefijos de *Conventional Commits*: «chore» para la configuración del repositorio, «feat» para cada funcionalidad, «style» para el diseño y «docs» para la documentación. El orden refleja cómo se construyó el sistema: primero la base de datos y la conexión, después la estructura MVC, la consulta, el formulario, los estilos y el README. Después de esta captura se agregó un último commit con este informe en la carpeta docs/.

4. ENLACES DEL PROYECTO

Recurso	Enlace o ubicación
Repositorio público en GitHub	https://github.com/glucas/AI3_DisenoWeb_GeorgeLucas
Instrucciones de despliegue	README.md (raíz del repositorio)
Script DDL	sql/db_sistema_web.sql
Informe	docs/AI3_Informe_GeorgeLucas.pdf

REFERENCIAS

OWASP Foundation. (s. f.). *Cross Site Scripting Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

OWASP Foundation. (s. f.). *Query Parameterization Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

PHP Group. (s. f.). *Choosing an API*. https://www.php.net/mysqlinfo.api.choosing.php

PHP Group. (s. f.). *PDO::prepare*. https://www.php.net/manual/es/pdo.prepare.php

PHP.Watch. (s. f.). *PHP 8.0: PDO: Default error mode set to exceptions*. https://php.watch/versions/8.0/PDO-default-errmode

PHP.Watch. (s. f.). *PHP 8.1: MySQLi: Default error mode set to exceptions*. https://php.watch/versions/8.1/mysqli-error-mode